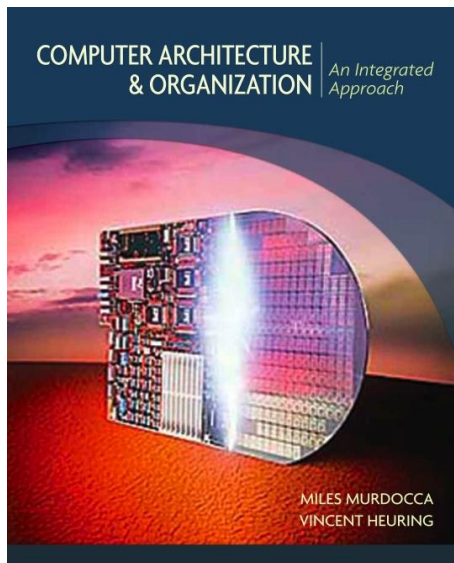


Organización de las Computadoras

Carolina Chaves Garcia



**Puertos
(UART)**

Tipos de puertos

Tipos de puertos

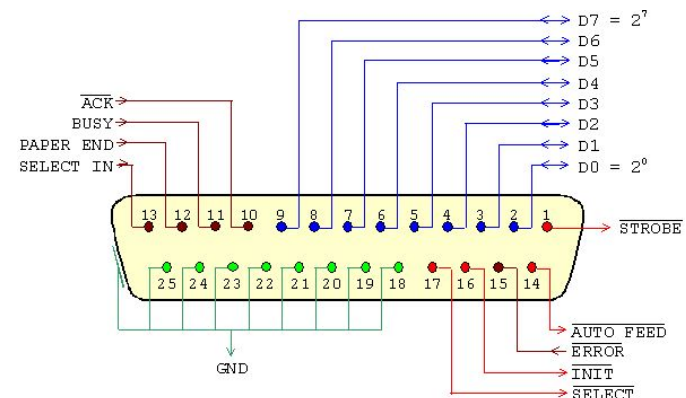
Puerto Paralelo

- Transmite múltiples bits simultáneamente por diferentes cables.
- Características ESP32-C3:
 - No tiene puerto paralelo nativo
 - Se puede emular con múltiples pines GPIO
- Ejemplo conceptual - Control paralelo de 8 LEDs

```
.equ LED0, 2
.equ LED1, 3
.equ LED2, 4
.equ LED3, 5
.equ LED4, 6
.equ LED5, 7
.equ LED6, 8
.equ LED7, 9
```

Configurar pines como salida

```
li a0, (1 << LED0) | (1 << LED1) | (1 << LED2) | (1 << LED3) | \
      (1 << LED4) | (1 << LED5) | (1 << LED6) | (1 << LED7)
call gpio_config_output
```



Tipos de puertos

Puerto Serie (UART)

- Transmite un bit a la vez por un único cable. Transfiere datos bit a bit secuencialmente.
- Transmisión de datos bit a bit a través de un único canal de comunicación.
- **Ventajas para ESP32-C3:**
 - Bajo consumo de pines (solo 2: TX y RX)
 - Distancias mayores
 - Menos interferencia
 - Menos cables
- **ESP32-C3: Tiene UARTs integrados (UART0, UART1)**
- **UART0 - Programación/Depuración**
 - USB nativo del ESP32-C3 (no necesita conversor externo)
 - Conectado internamente al chip USB-serial integrado
 - NO usa pines GPIO
 - Limitado para nuestros proyectos, solo podemos recibir y enviar datos
 - Se usa para:
 - Flashear el firmware
 - Monitor serial (printf, debug)

Tipos de puertos

Puerto Serie (UART)

- **UART1 - UART de Usuario/General**

- Sí usa pines GPIO (configurables)
- Necesita conversor USB-serial externo para conectar a PC
- Más libertad para aplicaciones
- Configuramos TODO desde cero
- Sin interferencias del sistema
- Baud rate personalizado
- Formato personalizado
- Interrupciones personalizadas

- **Baudrate configurables:**

- 9600
- 115200
- Etc

- **Registros para controlar:**

- Tx : Transmisión
- Rx: Recepción

Tipos de puertos

USB (Universal Serial Bus) y Firewire

- **Soporte USB Serial/JTAG integrado**
 - Tiene controlador USB integrado
 - Puede funcionar como dispositivo USB
 - Requiere configuración compleja
 - Ventajas:
 - Alimentación y datos
 - Hot-plug (conexión en caliente)
 - Alta velocidad

- **Firewire (IEEE 1394):**
 - Interfaz serial de alta velocidad
 - ESP32-C3: No compatible directamente
 - Uso actual: Reemplazado por USB y Thunderbolt

Tipos de puertos

Comparación en el ESP32-C3

Característica	Paralelo	Serie(UART)	USB
Pines requeridos	8+	2	2
Velocidad	Media	Media-alta	Alta
Complejidad	Media	Baja	Alta
Distancia	Corta	Media	Corta
Uso en ESP32-C3	Corta	Nativo	Nativo

- **Por qué UART es Ideal para Nuestras Prácticas**
 - Configuración simple - pocos registros
 - Operación en tiempo real - sin overhead del sistema
 - Control directo del hardware - ideal para assembler
 - Compatible con PC - mediante conversor USB-serial

Introducción a la comunicación serial

Comunicación Síncrona vs Asíncrona

- Síncrona (SPI, I2C):
 - Reloj compartido entre dispositivos
 - Mayor velocidad
 - Más compleja
- Asíncrona (UART): ← NUESTRO ENFOQUE
 - Sin reloj compartido
 - Usa baud rate pre-acordado
 - Más simple de implementar

Simplex, Half-Duplex, Full-Duplex

- Son modos de comunicación que definen la dirección del flujo de datos entre dos dispositivos.
- Carreteras vs Calles - direccionalidad del tráfico
- Simplex (Unidireccional):
 - Comunicación en UNA SOLA dirección
 - Un dispositivo transmite y el otro recibe

Tipos de Comunicación Serial

- Half-Duplex (Semidúplex)
 - Comunicación en AMBAS direcciones
 - Pero NO simultáneamente
- Full-Duplex (Dúplex Completo)
 - Comunicación en AMBAS direcciones
 - SIMULTÁNEAMENTE
- UART en ESP32-C3 es FULL-DUPLEX
 - Puede transmitir y recibir simultáneamente
 - UART_TX, GPIO21 # Transmisión
 - UART_RX, GPIO20 # Recepción

Protocolo UART - Estructura de la Trama

Protocolo UART - Estructura de la Trama

- **Ejemplo Transmisión del carácter 'A' (ASCII 65 = 0x41)**

- 0x41 = 01000001 = 'A'

1	0	0	1	0	0	0	0	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

IDLE	Start Bit Low	D7	D6	D5	D4	D3	D2	D1	D0	Stop Bit High	IDLE
------	------------------	----	----	----	----	----	----	----	----	------------------	------

- **Ejemplo Transmisión del carácter 'E'**

- 0x45 = 01000101 = 'E'

1	0	0	1	0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

IDLE	Start Bit Low	D7	D6	D5	D4	D3	D2	D1	D0	Stop Bit High	IDLE
------	------------------	----	----	----	----	----	----	----	----	------------------	------

Parámetros de Configuración UART

Parámetros de Configuración UART

- Baud Rate (Velocidad)
 - Bits por segundo que se transmiten.
 - Cálculo del divisor:
 - $\text{CLKDIV} = (\text{Clock_Frecuencia}) / (\text{Baud_Rate} * 16)$
- Configuración de Bits
 - Configuración típica:
 - 8 bits de datos
 - Sin bit de paridad (N)
 - 1 bit de stop
 - Control de flujo: ninguno
- Registros importantes
- UART_FIFO_REG 0x00 Buffer de datos
- UART_STATUS_REG 0x1C Estado de la UART
- UART_CLKDIV_REG 0x14 Configuración de baud rate
- UART_CONF0_REG 0x20 Configuración general

Configuración del puerto UART

Configuración del puerto UART

UART0

- Usaremos el UART0.
- **NO** lo re-configuraremos, sólo lo usaremos
- **UART0 YA ESTÁ CONFIGURADO** por el sistema:
 - Baud rate: 115200
 - Configuración: 8N1
 - Listo para usar
- Nosotros sólo: **ENVIAMOS** y **RECIBIMOS** datos, **SIN** tocar la configuración
- Compartimos el UART0:
 - Nuestro código Y el sistema comparten UART0:
 - printf() se mezcla con tus mensajes
 - Monitor serial recibe todo mezclado
 - Pero funciona perfectamente para aprendizaje

Ejemplos

Envío y recepción de datos por serie

- Vamos a tener tres funciones:
 - .global uart_send
 - .global uart_receive
 - .global uart_puts
- Definición registros UART0 ✓
 - UART_BASE
 - UART_CLKDIV_REG
 - UART_FIFO_REG
 - UART_STATUS_REG
 - UART_CONF0_REG
- Enviar un byte: Parámetro: a0 = carácter a enviar
 - Función uart_send
 - Esperar a que el buffer de TX esté listo UART_STATUS_REG ✓
 - Función uart_send_wait
 - Leer estado del UART ✓
 - Verificar bit TX_FULL ✓
 - Si está lleno, esperar en la función uart_send_wait ✓
 - Escribir caracter (usar sw en lugar de sb) UART_FIFO_REG ✓

Envío y recepción de datos por serie

- Recibir un byte: Retorna: a0 = carácter recibido (0 si no hay dato)
 - Función uart_receive:
 - Verificar si hay datos disponibles UART_STATUS_REG ✓
 - Leer estado ✓
 - Verificar bit RX_EMPTY (bit 30) ✓
 - Si está vacío, retornar a uart_no_data ✓
 - Leer carácter UART_FIFO_REG ✓
 - Función uart_no_data:
 - Retornar 0 si no hay datos ✓
- Enviar string: a0 = dirección del string
 - Función uart_puts:
 - Reservar en la pila 12 bytes ✓
 - Guardar el contenido del registro ra (Return Address) en la parte superior de la pila ✓
 - Guardar el valor del registro s0 en la pila, 4 bytes debajo de sp ✓
 - Guardar el valor del registro s1 en la pila 8 bytes debajo de sp ✓
 - Guardar dirección del string en s0 ✓
 - Inicializar el contador ✓

Envío y recepción de datos por serie

○ Función puts_loop:

- Leer carácter ✓
- Terminar si es NULL ✓
- Enviar carácter ✓
- Siguiente carácter ✓
- Incrementar contador ✓

○ Función puts_end:

- Carga de la pila (desde $sp + 8$) el valor original del registro s1 ✓
- Restaura el valor original del registro s0 desde la pila ✓
- Restaura el valor del registro ra (dirección de retorno) desde la pila ✓
- Libera los 12 bytes de la pila que fueron reservados al inicio ✓

Envío y recepción de datos por serie

```
void example_usage(void)
{
    // 1. Enviar string completo
    uart_puts("Iniciando sistema...\r\n");

    // 2. Enviar caracteres individuales
    uart_send('H');
    uart_send('O');
    uart_send('L');
    uart_send('A');
    uart_send('\r');
    uart_send('\n');

    // 3. Recibir datos
    while(1) {
        char data = uart_receive();
        if(data != 0) {
            // Procesar dato recibido
            if(data == 'q') break;

            // Hacer eco
            uart_send(data);
        }
    }
}
```

Ejemplos

Comunicación con LEDs/sensores.

- '1' → Enciende LED1 (GPIO2)
- '2' → Enciende LED2 (GPIO3)
- '0' → Apaga LEDs
- Cualquier letra → Eco

- Crear constantes: ✓
 - UART
 - GPIO
- PROGRAMA PRINCIPAL
 - app_main ✓
 - Configurar GPIO para LEDs, usando un llamado a la función configurar_gpio
 - Enviar mensaje de inicio por UART0, llamando la función uart_puts
 - main_loop: Intentar recibir carácter por UART0 ✓
 - Llamar la función uart_receive
 - Si no hay dato, continuar
 - Procesar comando recibido, llamando a la función procesar_comando

Comunicación con LEDs/sensores.

- CONFIGURACIÓN GPIO ✓
 - configurar_gpio
 - Como lo hemos realizado siempre
- CONTROL LEDs ✓
 - encender_led
 - apagar_led
- PROCESAMIENTO COMANDOS
 - procesar_comando
 - Reservar 8 bytes en la pila
 - Guardar el contenido del registro ra (Return Address) en la parte superior de la pila
 - Guardar el valor del registro a0 en la pila, 4 bytes debajo de sp = Guardar el carácter
 - Mostrar comando recibido llamando a uart_send
 - Procesar comando, recuperando el carácter. Carga desde la pila (posición sp + 4) un valor en a0
 - Carga el valor ASCII del carácter '1' en el registro t0
 - Compara si el carácter recuperado (a0) es '1'
 - Si es igual, salta a la función comando_led1, donde probablemente se enciende el LED 1. ✓

Ejemplos

Comunicación con LEDs/sensores.

- Carga el valor ASCII del carácter '2' en t0
- Si el carácter es '2', salta a la función comando_led2, para encender el LED 2. ✓
- Carga el valor ASCII de '0' en t0.
- Si el carácter es '0', salta a la función comando_apagar.
- Si no se cumplió ninguna condición anterior, se carga la dirección de una cadena (mensaje_desconocido) en a0
- Llamar a uart_puts para imprimir el mensaje
- comando_led1 ✓
 - Encender LED1
 - Cargar el mensaje mensaje_led1_on
 - Llamar a uart_puts para imprimir el mensaje
 - Saltar a procesar_fin
- comando_led2 ✓
 - Encender LED2
 - Cargar el mensaje mensaje_led2_on
 - Llamar a uart_puts para imprimir el mensaje
 - Saltar a procesar_fin

Ejemplos

Comunicación con LEDs/sensores.

- comando_apagar ✓
 - Apagar LEDs
 - Cargar el mensaje mensaje_leds_off
 - Llamar a uart_puts para imprimir el mensaje
 - Saltar a procesar_fin
- procesar_fin ✓
 - Guardar en memoria
 - Liberar espacio de la pila
- mensaje_inicio:
 .string "\r\n=== CONTROL LEDs UART0 ===\r\n1=Led1, 2=Led2, 0=Apagar\r\n> "
- mensaje_led1_on:
 .string "LED1 ON\r\n> "
- mensaje_led2_on:
 .string "LED2 ON\r\n> "
- mensaje_leds_off:
 .string "LEDs OFF\r\n> "
- mensaje_desconocido:
 .string "? Comando invalido\r\n> "

Ejemplos

Comunicación con LEDs/sensores.

```
void app_main(void) {  
    // Simplemente llamar al código assembler  
    printf("Iniciando sistema desde C...\r\n");  
    app_main();  
}
```

Gracias